AegisNet: A Self-Explaining Edge-AI Neural Network for Cyber-Threat and Server-Anomaly Detection

Rushi, Analyst

October 2026

1. Abstract

AegisNet is a small neural-network system (3,867 parameters, 34 KB) written from scratch in NumPy. It watches 13 server metrics every minute, detects both known and never-seen cyber-attacks, explains which metrics caused each alert, and passes a safe action plan to a human-supervised "Guardian" agent. It combines an autoencoder (trained on normal behaviour only) with a classifier (trained on labelled attacks) into a "Dual-Brain" detector. On 10 unseen days of simulated telemetry it detected 37 of 37 attack episodes, with 98.5% attack-minute recall, a 0.98% false-alarm rate and a fused ROC-AUC of 0.995. In a zero-day experiment where ransomware was hidden from training, a classifier alone detected 0% of it, while AegisNet detected 100% and flagged it as a new pattern instead of mislabelling it. The project uses only free tools, needs no GPU, and is verified by 12 automated tests.

2. Introduction and industry context (October 2026)

The AI field in October 2026 is defined by four things.

- **A crowded model race.** Industry trackers report that the frontier is shared by Anthropic's Claude Opus 5.5, OpenAI's GPT-6.x family and Google's Gemini 4 (launched in early October 2026), while open-weight models such as DeepSeek V4.1, Qwen3.8 and Kimi K3 have moved close to the frontier.
- **Agentic AI is the main enterprise trend.** Gartner is reported to predict that 40% of enterprise applications will embed task-specific AI agents by the end of 2026, up from under 5% in 2025.
- **A production gap.** One 2026 market study reports that 79% of enterprises say they have adopted AI agents but only 11% run them in production. McKinsey's 2026 survey finds that nearly nine in ten organisations use AI regularly, while scaling across the enterprise is still in progress.
- **Small models are gaining ground.** Gartner is reported to expect that by 2027 organisations will use small, task-specific models at least three times more than general-purpose LLMs, because they are cheaper, private and easier to govern.

The figures above come from industry trackers and surveys published in 2026 and should be re-checked against the original sources before being quoted in a formal submission.

The lesson for engineers is that the hard part is no longer producing a clever model. It is shipping a model that is small, private, explainable, monitored and safe to act on. AegisNet is designed around exactly those requirements, applied to IT security operations.

3. Problem statement

IT and security teams receive thousands of metrics and alerts. Rule-based thresholds miss new attacks, supervised classifiers cannot recognise attacks they were never trained on, and black-box scores do not tell an on-call engineer why they were paged. Cloud-hosted giant models add cost and send sensitive telemetry outside the network.

The problem: build a small, local neural-network system that (1) detects known and unknown attacks from server telemetry, (2) explains each alert, (3) acts only within safe, human-approved limits, and (4) warns when it becomes stale.

4. Objectives

1. Implement a neural network engine from scratch (forward pass, back-propagation, Adam) so every step can be explained and tested.
2. Detect six attack families and benign traffic surges with a low false-alarm rate.
3. Catch an attack family the model has never seen (zero-day behaviour).
4. Explain every alert by naming the responsible metrics.
5. Provide a Guardian agent with bounded autonomy and an audit log.
6. Monitor data drift to decide when to retrain.
7. Keep the model small enough for edge devices, using only free tools.

5. Scope and assumptions

- Telemetry is **simulated** so the project is offline, reproducible and fully labelled. Results on real traffic will be lower.
- Guardian actions are **simulated** and written to an audit log. They are not connected to a real firewall.
- The system analyses one host's metrics at one-minute resolution.

6. System architecture

Server telemetry (13 metrics per minute) goes to the feature engine (26 inputs). The 26 inputs feed two networks in parallel: Brain 1 (autoencoder) and Brain 2 (classifier). Both feed the fusion step, which produces a risk score and a label. The fusion output goes to three modules: Explain, Guardian agent and Drift watchdog. A diagram is available in `docs/ARCHITECTURE.md`.

Component	File	Role
Neural engine	aegisnet/nn.py	Dense layers, tanh/ReLU, MSE and cross-entropy losses, Adam, cosine learning-rate decay

Component	File	Role
Simulator and features	aegisnet/data.py	13-metric telemetry, 6 attacks plus benign flash-crowds, 26-dimension features
Dual-Brain model	aegisnet/models.py	Autoencoder, classifier, fusion, signature check, save and load
Explain, Guardian, Drift	aegisnet/explain.py	Per-metric explanation, incident builder, playbook agent, PSI drift monitor
Report	aegisnet/report.py	Charts and a single-file HTML report
Pipeline	run_all.py	One-click train, evaluate, zero-day test, drift test, report
API	api.py	Flask REST service
Tests	tests/test_aegisnet.py	12 automated tests

7. Methodology

7.1 Data simulation

Each minute produces 13 metrics: CPU %, memory %, disk I/O, inbound and outbound Mbps, open connections, failed logins, unique destination ports, average packet size, DNS queries, HTTP 5xx %, process count and file writes. A daily load cycle and a shared burst factor make the metrics correlated like a real server.

Six attack families are injected as episodes of 8 to 29 minutes. Each episode has a strength between 40% and 100% and ramps up over three minutes, so weak attacks overlap with normal behaviour.

Attack	Tell-tale metrics
Brute-force	failed logins and connections up
DDoS	inbound traffic, connections and HTTP 5xx up, small packets
Port scan	unique ports far up, tiny packets
Exfiltration	outbound traffic and DNS queries up
Cryptominer	CPU and process count up
Ransomware	file writes and disk I/O far up

Flash crowds are benign traffic surges (inbound traffic and connections rise, CPU rises slightly) that resemble a DDoS. They are labelled normal and included in the data to test for false alarms.

Datasets: 28,800 minutes (20 days) of normal data to train Brain 1; 28,800 minutes of labelled data to train Brain 2; 14,400 minutes (10 days) of unseen test data containing 37 attack episodes and 685 attack minutes.

7.2 Feature engineering

Each metric is transformed with log(1 + x) to tame heavy tails. For every metric a second feature stores the change versus the average of the previous five minutes. This gives 13 level features plus 13 change features, which is 26 inputs. All inputs are standardised using the mean and standard deviation of the normal training data and clipped to the range minus 20 to plus 20.

7.3 Neural-network engine

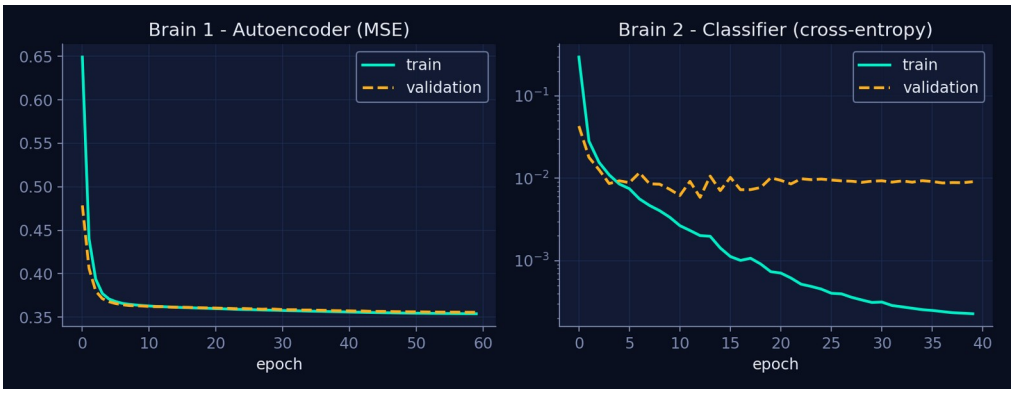
For a layer with input x, weights W and bias b, the forward pass is z = xW + b, followed by an activation: tanh(z), ReLU(z) = max(z, 0) or the identity. Back-propagation applies the chain rule layer by layer. For tanh the local gradient is 1 minus tanh squared, and for ReLU it is 1 where z is positive.

- **Autoencoder loss:** mean squared error, L = mean((output minus input) squared).
- **Classifier loss:** softmax cross-entropy with class weights, L = minus the weighted mean of log p(correct class).
- **Optimiser:** Adam with beta1 = 0.9, beta2 = 0.999, small weight decay, and a cosine learning-rate schedule from 3e-3 down to 10% of that value.
- **Initialisation:** He initialisation for ReLU layers and Xavier initialisation for tanh and linear layers.

Correctness is proven by a gradient-check test that compares analytic gradients with numerical gradients for both loss types. A second test confirms the engine learns the XOR problem.

7.4 Brain 1: autoencoder (unknown threats)

Architecture 26, 18, 6, 18, 26 (1,220 parameters). It is trained only on normal data, split 85% training and 15% validation. The reconstruction error of a minute is how badly the network can rebuild that minute. The alert threshold is the 99.5th percentile of validation errors, so the target false-alarm rate on normal data is about 0.5%. The anomaly score is the error divided by the threshold, so a score above 1 means “unusual”.



Training curves for both brains

7.5 Brain 2: classifier (known attacks)

Architecture 26, 48, 24, 7 (2,647 parameters) with seven outputs: normal plus six attack families. Class weights are the square root of the inverse class frequency, which helps rare attacks without over-weighting them.

7.6 Fusion and decision logic

1. Brain 1 raises a flag if its score is above 1. Brain 2 raises a flag if the probability of any attack is above 0.5.
2. A minute is a raw alert if either brain flags it.
3. The risk score (0 to 100) is the larger of a sigmoid of the autoencoder score and 100 times the attack probability.
4. Persistence: a final alert needs a raw alert in at least two of the last three minutes, or a risk of 90 or more. This removes one-minute blips.
5. Labelling: see the signature check below.

7.7 Signature-consistency check (the zero-day safety net)

For each known attack, the model stores a prototype: the average share of reconstruction error contributed by each metric during that attack in training. For a new alert, the classifier's suggested family is accepted only if the minute's error pattern is similar (cosine similarity above a per-family threshold learned from training data) to that family's prototype. Otherwise the alert is labelled **unknown_anomaly**. This prevents a never-seen attack from being confidently given the wrong name.

7.8 Explainability

The autoencoder's error is split per metric. The metrics that carry the largest share of the error (at least 5% each, at most three) are reported with their observed value and the value the autoencoder expected. The "expected" value is the network's reconstruction, so it is a model estimate rather than a ground-truth baseline.



Which metrics caused each alert

7.9 Guardian agent (bounded autonomy)

Consecutive alert minutes are merged into an incident. A playbook maps each incident type to a severity and a list of actions, each marked either safe or needing human approval. Safe actions (snapshots, tickets, paging, higher logging) run automatically. Risky actions (blocking IPs, isolating a host, quarantine) wait until a human approves. Every action is written to an audit log. Actions are simulated in this version.

7.10 Drift watchdog

The Population Stability Index compares the live distribution of each metric with the training distribution using ten quantile bins. PSI below 0.1 is stable, 0.1 to 0.25 is a warning, and above 0.25 means retrain.

8. Implementation

Item	Detail
Language	Python 3
Libraries (all free)	NumPy, pandas, Matplotlib, scikit-learn (metrics and Isolation-Forest baseline only), Flask (API only)
Training time	about 4 seconds for both brains on one CPU core
Full pipeline	about 11 seconds, including the zero-day experiment
Model size	34 KB (3,867 parameters)
Inference	about 0.004 ms per minute-window on one CPU core

How to run

1. Install: `pip install -r requirements.txt`
2. Train, evaluate and build the report: `python run_all.py` (use `--quick` for a 3-second smoke test), then open `outputs/report.html`.
3. Run the tests: `python -m unittest discover -s tests -v`
4. Start the API: `python api.py`, then send `sample_request.json` to `http://127.0.0.1:5000/predict`.

9. Testing and results

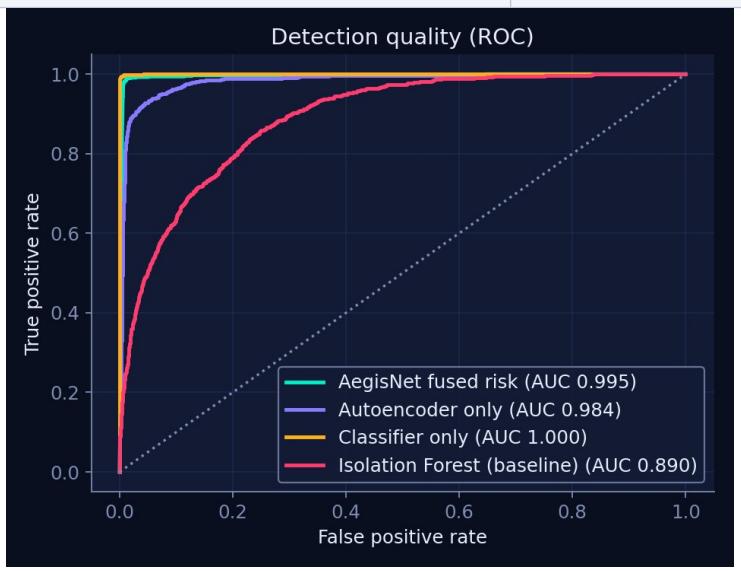
9.1 Automated tests (12 of 12 pass)

Test	What it proves
Gradient check (MSE and cross-entropy)	Back-propagation matches numerical gradients
Learns XOR	The engine can learn a non-linear function
Simulator	Deterministic output, correct columns, no negative values, valid labels
Detection quality	Fused AUC above 0.97, recall above 90%, false-alarm rate below 3%
Every attack family detected	Alert recall above 80% for each family
Save and load	Reloaded model gives identical risk scores and labels
Explanation	The top metric of each incident matches the attack's real signature
Guardian	Risky actions wait for approval and are audit-logged
Zero-day	Unseen ransomware is flagged and never mislabelled as a known attack
Drift watchdog	Stable data reads STABLE, shifted data reads DRIFT on the right metric
Edge speed	Under 1 ms per minute-window
REST API	Health check, input validation, DDoS alert with incident plan, calm window with no alert

The project was also verified by unzipping the delivered package into a clean folder, running the full pipeline and tests, and calling the live API.

9.2 Detection quality (10 unseen days)

Score	ROC-AUC
Classifier only	0.9999
AegisNet fused risk	0.995
Autoencoder only	0.984
Isolation Forest baseline	0.890



ROC curves

Decision rule	Precision	Recall	False-alarm rate	F1
Raw (either brain flags a minute)	78.7%	99.0%	1.33%	0.877
Final (two-of-three persistence)	83.3%	98.5%	0.98%	0.903

Attack minutes are only about 5% of the test set, so even a 1% false-alarm rate lowers precision to 83%.



One day of live monitoring

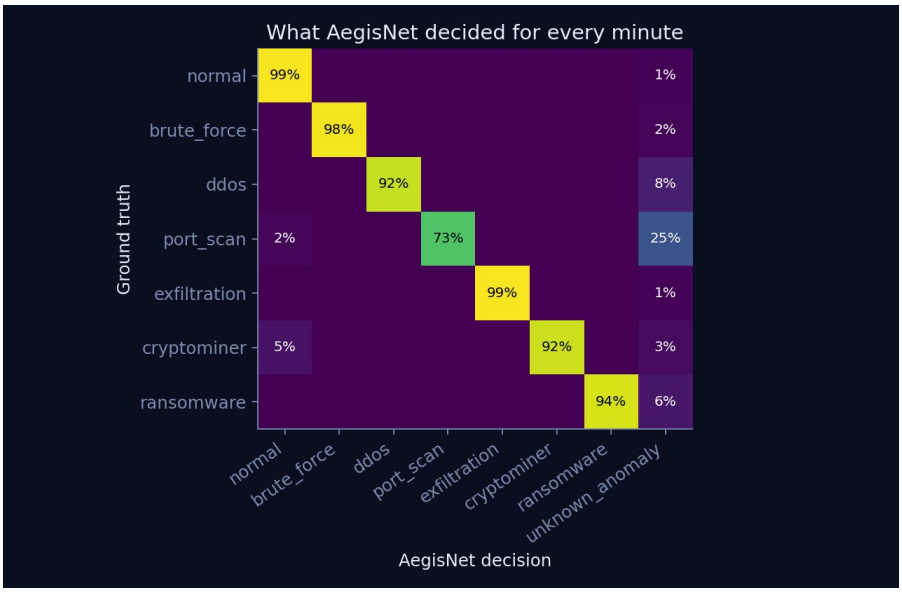
9.3 Incident-level results

Measure	Value
True attack episodes	37
Episodes detected	37 (100%)
Mean time to detect	0.2 minutes (maximum 1 minute)
Incidents raised	40
Incidents with no real attack behind them	3

9.4 Per-attack results

Attack	Test minutes	Alert recall	Named correctly
Brute-force	63	100.0%	98.4%
DDoS	169	100.0%	91.7%
Port scan	59	98.3%	72.9%
Exfiltration	100	100.0%	99.0%
Cryptominer	181	95.0%	92.3%
Ransomware	113	100.0%	93.8%

Most minutes that were not named correctly were labelled unknown_anomaly, which is still an alert. Port-scan naming is the weakest result.



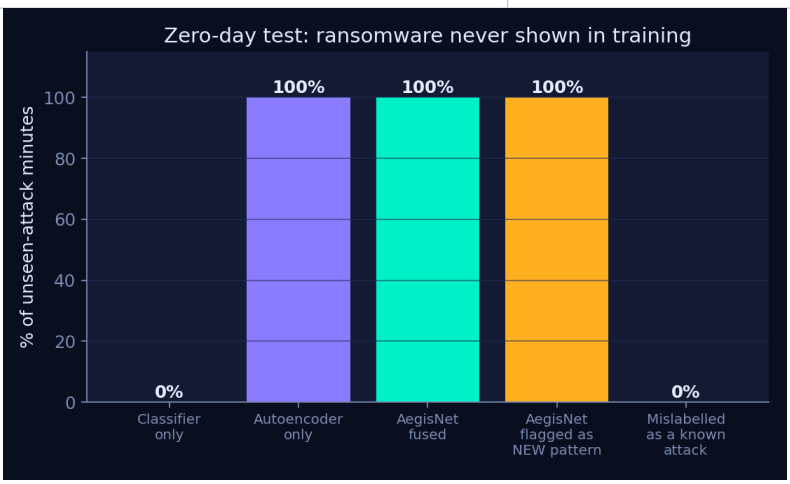
Decision matrix

9.5 Zero-day experiment

Ransomware was removed from the classifier's training data and then shown in the test data.

System	Share of ransomware minutes
Classifier alone flags it	0%
Autoencoder alone flags it	100%

System	Share of ransomware minutes
AegisNet flags it	100%
AegisNet labels it unknown_anomaly (new pattern)	100%
AegisNet mislabels it as a known attack	0%

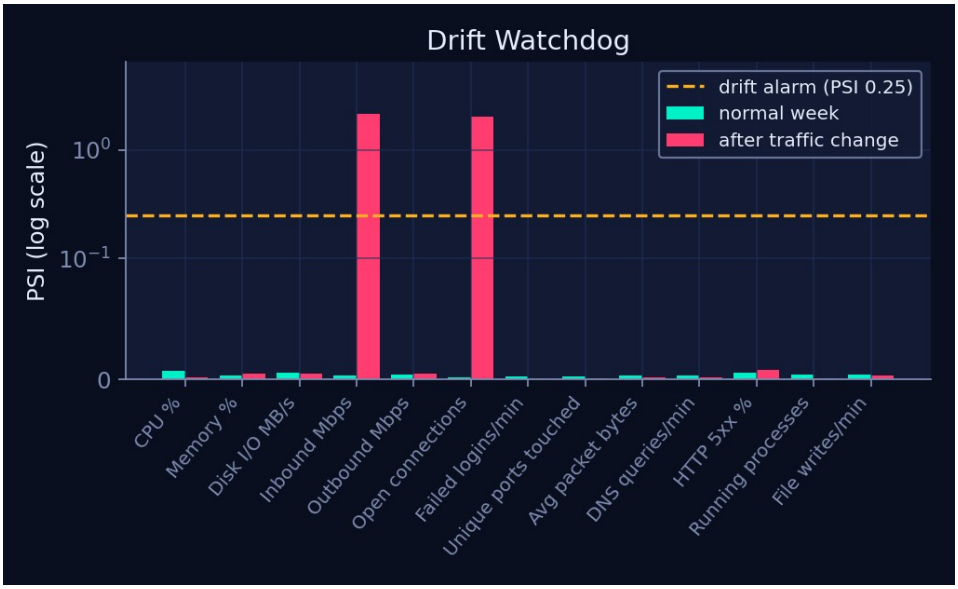


Zero-day experiment

This shows why the Dual-Brain design is needed. The classifier is the most accurate on known attacks, but it cannot recognise what it was never shown. The autoencoder only needs to know what normal looks like.

9.6 Drift test

A fresh normal week gave a worst PSI of 0.007 (STABLE). After shifting inbound traffic by 1.6 times and connections by 1.5 times, the worst PSI was 2.15 on inbound traffic (DRIFT, retrain).



Drift watchdog

10. Discussion and limitations

- **Simulated data.** The attack signatures are cleaner than in real networks, which is why the classifier reaches an AUC close to 1.0. Real datasets such as CICIDS2017 or NSL-KDD, or production logs, are the essential next test.
- **Classifier versus fusion.** On known attacks the classifier alone ranks slightly higher (0.9999 against 0.995). The fused system trades a little ranking quality for the ability to catch unseen attacks.
- **Explanations are model-based.** The expected values come from the autoencoder's reconstruction and are pulled toward the anomaly when it is large.
- **Guardian is simulated.** No real firewall or ticketing system is called.
- **Single host, one-minute resolution.** Multi-host correlation is not covered.
- **Reproducibility.** Seeds are fixed, so results reproduce exactly on the same software versions. Small differences may appear with other NumPy versions, and latency depends on the machine.

11. Future scope

1. Replace the simulator with CICIDS2017, NSL-KDD or real logs through a loader that returns the same columns.
2. Add a local language model (for example through Ollama) to write plain-language incident stories from the structured explanation.
3. Connect the Guardian to a real firewall and ticketing API, keeping the human-approval rule.
4. Export the networks to ONNX for deployment on edge devices.
5. Train federated models across several servers without sharing raw telemetry.
6. Add a live dashboard in Streamlit or Grafana.

12. Conclusion

AegisNet shows that a very small, fully local neural network can detect known and unknown attacks, explain its decisions, act only within human-approved limits and tell its owner when it has gone stale. It reaches a fused ROC-AUC of 0.995, detects all 37 test attacks within about one minute, and catches a completely unseen attack family that a classifier alone misses entirely. It does this in 34 KB with free tools. The main limitation is that the data is simulated, and the clearest next step is to validate on real traffic.

13. References

Industry and market context (2026)

- McKinsey & Company, "The state of AI in 2026" (global survey).
- Gartner predictions on agentic AI and small language models, as reported by MachineLearningMastery ("7 Agentic AI Trends to Watch in 2026"), Firecrawl ("Top 15 Agentic AI Trends to Watch in 2026") and Svitla ("Agentic AI Market Trends 2025-2026").
- CNBC, "Can Google's new model really catch up to OpenAI and Anthropic at the frontier?", 2 October 2026.
- Open-weight and frontier model trackers (Slash Digital, teamday.ai, local-ai-zone), September to October 2026.

Technical references

- D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization”, ICLR 2015.
- G. E. Hinton and R. R. Salakhutdinov, “Reducing the Dimensionality of Data with Neural Networks”, Science, 2006.
- F. T. Liu, K. M. Ting and Z.-H. Zhou, “Isolation Forest”, IEEE ICDM, 2008.
- I. Sharafaldin, A. H. Lashkari and A. A. Ghorbani, “Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization”, ICISSP, 2018 (CICIDS2017).
- M. Tavallaee et al., “A Detailed Analysis of the KDD CUP 99 Data Set”, IEEE CISDA, 2009 (NSL-KDD).

Viva preparation questions are in `docs/VIVA_QA.md`.